

Building a Rest API with Python3

Prerequisites

Python Concepts

Before starting, you should know:

Data Types

```
name = "Laptop"  
price = 500.50  
available = True
```

OOP / Classes

```
class Product:  
    pass
```

Inheritance

```
class Vehicle:  
    pass
```

```
class Car(Vehicle):  
    pass
```

Properties

```
class Employee:  
    @property  
    def salary(self):  
        return self._salary
```

Files & Pathlib

```
from pathlib import Path
```

```
file = Path("data.txt")
```

Regular Expressions

```
import re
```

```
re.search(r"\d+", "Price: 100")
```

3. Best Practices Used

PEP 8 Style Guide

```
def calculate_total():  
    pass
```

Type Hints

```
def add(a: int, b: int) -> int:  
    return a + b
```

mypy

Static type checker.

mypy .

Unit Testing

Using pytest.

```
def test_add():  
    assert 2 + 2 == 4
```

Poetry

Dependency management.

poetry install

4. Libraries Used

FastAPI

Create REST APIs.

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

SQLAlchemy

Database ORM.

```
from sqlalchemy import create_engine
```

PyYAML

Parse YAML files.

```
import yaml
```

dotenv

Load environment variables.

```
from dotenv import load_dotenv
```

pytest

Unit testing.

```
pytest
```

5. Project Setup

Download Project

<https://bit.ly/python3-rest-api>

Install Poetry

<https://bit.ly/installing-poetry>

Install Dependencies

```
poetry install
```

Run Project

```
poetry run python runserver.py
```

6. Database Design

The project stores events and categories.

Category Table

```
CREATE TABLE category (  
    id INTEGER PRIMARY KEY,  
    name VARCHAR NOT NULL UNIQUE  
);
```

Event Table

```
CREATE TABLE event (  
    id INTEGER PRIMARY KEY,  
    product_code VARCHAR NOT NULL UNIQUE,  
    date DATE NOT NULL,  
    price NUMERIC NOT NULL,  
    category_id INTEGER NOT NULL  
    REFERENCES category  
);
```

7. SQLAlchemy Models

Event Model

```
from sqlalchemy.orm import Mapped  
from sqlalchemy.orm import mapped_column  
from sqlalchemy.orm import relationship  
from sqlalchemy import ForeignKey
```

```
class DBEvent(Base):  
    __tablename__ = "event"
```

```
id: Mapped[int] = mapped_column(primary_key=True)
```

```
product_code: Mapped[str] = mapped_column(  
    unique=True  
)
```

```
date: Mapped[date]
```

```
price: Mapped[decimal.Decimal]
```

```
category_id: Mapped[int] = mapped_column(  
    ForeignKey("category.id")  
)
```

```
category: Mapped["DBCategory"] = relationship(  
    back_populates="events"  
)
```

Category Model

```
class DBCategory(Base):
```

```
    __tablename__ = "category"
```

```
id: Mapped[int] = mapped_column(  
    primary_key=True  
)
```

```
name: Mapped[str] = mapped_column(  
    unique=True  
)
```

```
events: Mapped[list["DBEvent"]] = relationship(  
    back_populates="category"  
)
```

8. Relationships

One category can contain many events.

Category

|

```
| 1
|
|-----> Many Events
```

Example:

```
sports = DBCategory(name="Sports")
```

```
event1 = DBEvent(
    product_code="SP100",
    price=100
)
```

```
event2 = DBEvent(
    product_code="SP200",
    price=200
)
```

```
sports.events.append(event1)
sports.events.append(event2)
```

9. Environment Configuration with dotenv

Install

```
pip install python-dotenv
```

.env File

```
DATABASE_URL="sqlite:///events.db"
```

Load Configuration

```
import os
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
DATABASE_URL = os.getenv(
    "DATABASE_URL"
)
```

10. Database Connection

```
from sqlalchemy import create_engine
```

```
engine = create_engine(  
    DATABASE_URL  
)
```

11. FastAPI Basics

Create Application

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

Root Endpoint

```
@app.get("/")  
def home():  
    return {"message": "Hello"}
```

12. REST API Endpoints

The course creates:

Get Single Event

```
@app.get("/events/{event_id}")  
def get_event(event_id: int):  
    pass
```

Get All Events

```
@app.get("/events")  
def get_events():  
    pass
```

13. Pydantic Schemas

FastAPI uses Pydantic for validation.

Event Schema

```
from pydantic import BaseModel
```

```
class EventSchema(BaseModel):  
    id: int  
    product_code: str  
    price: float
```

Example Response

```
{  
  "id": 1,  
  "product_code": "SP100",  
  "price": 99.99  
}
```

14. Unit Testing API

Using pytest.

Simple Test

```
def test_event():  
    assert 1 == 1
```

FastAPI Test Client

```
from fastapi.testclient import TestClient
```

```
client = TestClient(app)
```

Test Endpoint


```
def test_home():
    response = client.get("/")

    assert response.status_code == 200
```

15. Reading Content from Filesystem

Pathlib

Used for file discovery.

```
from pathlib import Path
```

```
folder = Path("data")
```

Get Files

```
files = folder.glob("*.yaml")
```

Example

```
for file in files:
    print(file)
```

16. Parsing Files with Regex

Example:

```
import re
```

```
text = "EVENT-100"
```

```
match = re.search(
    r"EVENT-\d+",
    text
)
```

17. Parsing YAML Files

